Day 170: Learning-Rate Finders

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 25, Day 3 | Complete study guide

Watch: Choosing the Learning Rate with LR Finder (Apache MXNet) | Competition Winning Learning Rates (Leslie Smith, MLconf)

Script: `day170_learning_rate_finder.py` · every number in this guide is copied by program from `day170_run_output.txt`, the real output of that script.

A learning-rate finder spends one short run on the most important setting: raise the learning rate a little every step and watch the loss. Today it is built from scratch and checked against real training. On least squares, where gradient descent provably diverges above 2 / (largest eigenvalue), the finder's lowest loss sat at about 5.54 times that limit, so its minimum is not a safe rate; the usual rule, one tenth of it, landed at 0.554 times the limit. On a network for UCI Adult that rule's suggestions trained to within +0.0088 of the best of 13 learning rates, for one finder run costing about 0.10 s against 38 s for the sweep. Two rules did not hold up: 'steepest slope' moved by a factor of 644.7 between seeds with Adam, and the finder did not follow a change of batch size.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

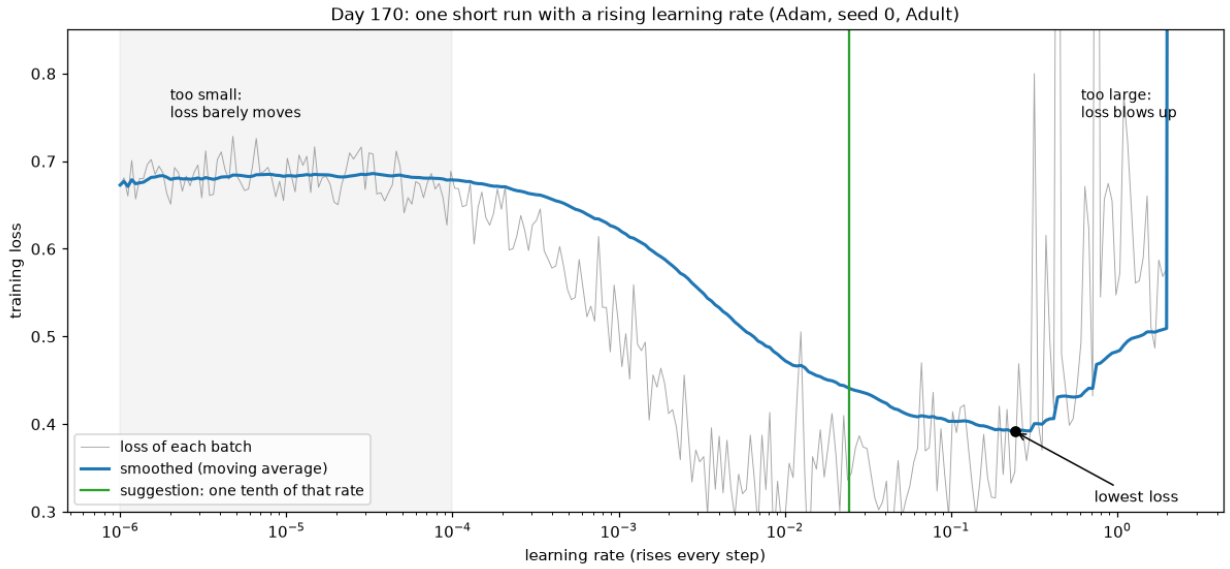
The **learning rate** is the size of each step a model takes against its gradient. Too small and training crawls; too large and the loss bounces or explodes. A **learning-rate finder** (L. Smith, 2015; popularised by fast.ai) starts a fresh model at a tiny rate and multiplies the rate by the same factor every step, here from 10^{-6} to 10 in 300 steps, recording the loss. It stops when the loss has blown up.

Reading the curve needs a rule. Two common ones: **one tenth of the rate at the lowest loss**, and **the steepest point**, where the loss falls fastest.

REAL-WORLD SCENARIO

Finding how fast you can drive a new road: start slowly, speed up a little every minute, and note when the car starts to skid. You would then drive well below that speed, not at it.

SYSTEM DIAGRAM



A real finder run (Adam, seed 0, on Adult). Grey: the loss of each batch; blue: its moving average; green: the suggested rate, one tenth of the rate at the lowest smoothed loss.

The network is Day 126's, imported unchanged: 87 inputs (Day 161's standardised numbers and one-hot categories), two ReLU layers of 64 and 32, one output, He initialisation and Day 37's Adam. Plain SGD and the log loss on logits are new today. It trains on Week 24's 24,130 fitting rows of UCI Adult and is scored on the 6,032 validation rows; the test file is not touched.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. A known truth: least squares

DEFINITION — start here, no prior knowledge assumed

For least squares, the loss is a bowl whose steepest curvature is the largest eigenvalue λ of $X^{T}X$ / n. Full-batch gradient descent with a constant rate converges when the rate is below $2/\lambda$ and diverges above it. This gives a known answer to compare the finder with.

The script checks the theory with constant rates just either side of the limit, then runs the finder from 1/1000 to 100 times the limit, without smoothing and with the usual smoothing.

REAL-WORLD SCENARIO

A ball rolling in a bowl: nudge it gently and it settles at the bottom; overcorrect each time by more than twice the right amount and it swings higher and higher.

MATHEMATICS

$w \leftarrow w - \eta\, X^T(Xw - y) / n$ *the error along the steepest direction is multiplied by* $|1 - \eta\lambda|$ *each step converges when* $|1 - \eta\lambda| < 1$, *that is* $\eta < 2/\lambda$ *finder:* $\eta_t = lo\,(hi/lo)^{t/(T-1)}$

IMPLEMENTATION

```
def lr_range(lo, hi, steps):
    return lo * (hi / lo) ** (np.arange(steps) / (steps - 1))
def lr_find(step, lrs, beta=0.98, blowup=4.0):
    raw, sm, avg, best = [], [], 0.0, np.inf
    for t, lr in enumerate(lrs):
        loss = step(lr); avg = beta * avg + (1 - beta) * loss
        s = avg / (1 - beta ** (t + 1))
        raw.append(loss); sm.append(s)
        if not np.isfinite(s) or s > blowup * best:
            break
        best = min(best, s)
    return np.asarray(lrs[:len(sm)]), np.array(raw), np.array(sm)
def suggest(lrs, sm, skip=10):
    g = np.gradient(sm, np.log(lrs))
    steep = lrs[skip + int(np.argmin(g[skip:-5]))] if len(sm) > skip + 6 else lrs[0]
    return dict(min_over_10=float(lrs[int(np.argmin(sm))] / 10), steepest=float(steep))
```

ACTUAL OUTPUT

```
  largest eigenvalue of X^T X / n: 3.5522; gradient descent diverges above 2 / 3.5522 = 0.5630
  constant rate 0.90 x the limit, 2000 steps: loss 0.05795
  constant rate 0.99 x the limit, 2000 steps: loss 0.05795
  constant rate 1.01 x the limit, 2000 steps: loss 9.08e+32
  constant rate 1.10 x the limit, 2000 steps: loss inf
  finder (smoothing 0.0): lowest loss at rate 4.94 x the limit; stopped at 5.87 x the limit; min / 10 suggests 0.494 x the
      limit
  finder (smoothing 0.98): lowest loss at rate 5.54 x the limit; stopped at 6.22 x the limit; min / 10 suggests 0.554 x the
      limit
Actual output from this lesson's run.
```

What it means

The theory holds exactly. The limit is 0.5630. At 0.99 x the limit the loss settled at 0.05795; at 1.01 x the limit it reached 9.08e+32.

The finder's minimum is far above the limit. Without smoothing its lowest loss came at 4.94 times the limit, with smoothing at 5.54, and it stopped near 6. Above the limit the error grows by only $|1 - \eta\lambda|$ per step, a little over 1 at first, and the rate keeps rising, so the explosion shows up only several steps (and several times the limit) later. A likely further reason, not tested here, is that only the steepest direction diverges at first while the loss in all the others is still falling.

That is why the rule divides by ten. One tenth of the minimum's rate is 0.494 and 0.554 times the limit: safe, and within a factor of two of the fastest stable rate.

WHY CHOOSE THIS?

A case with a provable answer shows what the finder's curve really measures.

WHY NOT THIS? (TRADEOFFS)

A network has no single limit; its curvature changes as it trains.

2. The finder on a neural network for Adult

DEFINITION — start here, no prior knowledge assumed

Five finder runs per optimiser, each with a different seed (initial weights and batch order), 300 steps of 128 rows: about one and a half epochs. Each run gives two suggestions.

The loss of a single batch is noisy, so the curve is smoothed with an exponential moving average (β = 0.98), corrected for starting at zero. The run stops when the smoothed loss exceeds four times its best.

REAL-WORLD SCENARIO

Testing a car's top speed on five different days: the wind changes each time. A rule that gives nearly the same answer every day is worth more than one that is sometimes better.

MATHEMATICS

$a_t = \beta\, a_{t-1} + (1-\beta)\, loss_t \quad smoothed_t = a_t / (1-\beta^{t+1}) \quad stop\ when\ smoothed > 4 \times best$

$min / 10 = (rate\ at\ the\ lowest\ smoothed\ loss) / 10 \quad steepest = argmin\ over\ t\ of\ d\ smoothed / d\ log(rate)$

ACTUAL OUTPUT

```
  network 87 -> 64 -> 32 -> 1, batches of 128; 300 steps from 1e-06 to 10
  Adam seed 0: 271 steps, stopped at 2.09e+00; lowest smoothed loss 0.3914 at 2.42e-01; min / 10 suggests 2.42e-02; steepest
      3.25e-03
  Adam seed 1: 300 steps, stopped at 1.00e+01; lowest smoothed loss 0.3824 at 3.54e-01; min / 10 suggests 3.54e-02; steepest
      1.04e+00
  Adam seed 2: 300 steps, stopped at 1.00e+01; lowest smoothed loss 0.3889 at 1.75e-01; min / 10 suggests 1.75e-02; steepest
      4.49e-03
  Adam seed 3: 261 steps, stopped at 1.22e+00; lowest smoothed loss 0.3871 at 2.85e-01; min / 10 suggests 2.85e-02; steepest
      8.38e-01
  Adam seed 4: 300 steps, stopped at 1.00e+01; lowest smoothed loss 0.3816 at 1.41e-01; min / 10 suggests 1.41e-02; steepest
      2.09e+00
  Adam min_over_10: suggestions span a factor of 2.5 over 5 seeds
  Adam steepest   : suggestions span a factor of 644.7 over 5 seeds
  SGD  seed 0: 299 steps, stopped at 9.48e+00; lowest smoothed loss 0.4283 at 6.86e+00; min / 10 suggests 6.86e-01; steepest
      9.33e-01
  SGD  seed 1: 300 steps, stopped at 1.00e+01; lowest smoothed loss 0.4326 at 6.50e+00; min / 10 suggests 6.50e-01; steepest
      1.85e-01
  SGD  seed 2: 300 steps, stopped at 1.00e+01; lowest smoothed loss 0.4423 at 5.24e+00; min / 10 suggests 5.24e-01; steepest
      3.94e-01
  SGD  seed 3: 300 steps, stopped at 1.00e+01; lowest smoothed loss 0.4425 at 4.00e+00; min / 10 suggests 4.00e-01; steepest
      4.16e-01
  SGD  seed 4: 300 steps, stopped at 1.00e+01; lowest smoothed loss 0.4364 at 4.70e+00; min / 10 suggests 4.70e-01; steepest
      1.75e-01
  SGD  min_over_10: suggestions span a factor of 1.7 over 5 seeds
  SGD  steepest   : suggestions span a factor of 5.3 over 5 seeds
Actual output from this lesson's run.
```

What it means

The min / 10 rule was steady: its suggestions spanned a factor of 2.5 over five seeds with Adam (1.41e-02 to 3.54e-02) and 1.7 with SGD (4.00e-01 to 6.86e-01). The steepest rule was not: with Adam it ranged from 3.25e-03 to 2.09e+00, a factor of 644.7. A likely reason, not tested here: the Adam curves are long and flat before they rise, so the steepest drop can come from a small wobble almost anywhere along them.

Only 3 of the 10 runs hit the blow-up test (Adam seeds 0 and 3, at 2.09e+00 and 1.22e+00, and SGD seed 0 at 9.48e+00); the rest ran to 10 with the loss raised but under four times its best. **The smoothing also shifts the curve.** With β = 0.98 the average spans about 50 steps, while the rate rises by a factor of about 1.055 per step (10^{7} over 300 steps), so the smoothed curve lags the raw losses by roughly 1.055^{50}, about 15 times in rate (diagram: the raw losses bottom out well left of the blue minimum). That arithmetic is a likely reason the minimum sits so high; it was not tested separately.

WHY CHOOSE THIS?

One run of about 1.5 epochs maps the whole range of rates from 10^{-6} to 10.

WHY NOT THIS? (TRADEOFFS)

The answer depends on the reading rule, the smoothing and the seed; read it with min / 10 and look at the curve.

3. The check: real training at 13 learning rates

DEFINITION — start here, no prior knowledge assumed

The finder only predicts. The check trains the network for real: 5 epochs at each of 13 rates from 10^{-5} to 10 (half a decade apart), 3 seeds each, scored on the validation rows. Then each finder run's two suggestions are trained for real with the same 3 seeds, so they are compared on equal terms.

A first draft compared one seed's run at a suggestion with the sweep's average over 2 seeds. That mixes the luck of one seed into the comparison, so it was replaced: everything now uses the same 3 seeds.

REAL-WORLD SCENARIO

Before trusting a quick blood-sugar meter, a clinic checks a batch of its readings against the laboratory's slow, careful test.

IMPLEMENTATION

```
def train_mlp(Xa, ya, Xv, yv, opt_cls, lr, seed, epochs=EPOCHS, batch=BATCH):
    sizes = [Xa.shape[1], *HIDDEN, 1]; acts = ["relu"] * len(HIDDEN) + ["linear"]
    net = d126.init_net(sizes, seed); opt = opt_cls(net, lr); rng = np.random.RandomState(seed +
        1)
    with np.errstate(all="ignore"):
        for _ in range(epochs):
            o = rng.permutation(len(Xa))
            for s in range(0, len(Xa), batch):
                A = d126.forward(net, Xa[o[s:s + batch]], acts)
                opt.step(net, d126.backward(net, A, acts, bce_logits(A[-1], ya[o[s:s +
                    batch]])[1])[0])
        v = bce_logits(d126.forward(net, Xv, acts)[-1], yv)[0]
    return v if np.isfinite(v) else np.inf
```

ACTUAL OUTPUT

```
  each run: 5 epochs on the 24130 fitting rows, log loss on the 6032 validation rows
  Adam: 1e-05 0.5669, 3e-05 0.4377, 1e-04 0.3459, 3e-04 0.3313, 1e-03 0.3288, 3e-03 0.3277, 1e-02 0.3298, 3e-02 0.3349,
      1e-01 0.3396, 3e-01 0.4326, 1e+00 0.5579, 3e+00 0.5506, 1e+01 3.0660
  Adam: best 3e-03 (0.3277); within 0.005 of it from 3e-04 to 1e-02; its 3 seeds there: 0.3271, 0.3271, 0.3288
  Adam: training at each finder run's min_over_10 suggestion (3 seeds each): 0.3313, 0.3365, 0.3298, 0.3321, 0.3286 (worst
      +0.0088 against the sweep's best)
  Adam: training at each finder run's steepest    suggestion (3 seeds each): 0.3278, 0.6278, 0.3272, 0.5514, 0.7767 (worst
      +0.4491 against the sweep's best)
  SGD: 1e-05 0.6725, 3e-05 0.6654, 1e-04 0.6459, 3e-04 0.6014, 1e-03 0.5243, 3e-03 0.4197, 1e-02 0.3583, 3e-02 0.3388, 1e-01
      0.3303, 3e-01 0.3330, 1e+00 0.3359, 3e+00 0.3641, 1e+01 diverged
  SGD: best 1e-01 (0.3303); within 0.005 of it from 1e-01 to 3e-01; its 3 seeds there: 0.3301, 0.3308, 0.3301
  SGD: training at each finder run's min_over_10 suggestion (3 seeds each): 0.3335, 0.3346, 0.3344, 0.3331, 0.3314 (worst
      +0.0043 against the sweep's best)
  SGD: training at each finder run's steepest    suggestion (3 seeds each): 0.3358, 0.3305, 0.3342, 0.3330, 0.3307 (worst
      +0.0055 against the sweep's best)
  (38 s for the sweeps and the checks)
Actual output from this lesson's run.
```

What it means

The sweep. Adam was best at 3e-03 (0.3277) and within 0.005 of that from 3e-04 to 1e-02: a wide, flat valley. SGD was best at 1e-01 (0.3303) and diverged at 10. Seeds at the best rate differed by about 0.002.

The min / 10 rule passed. Its suggestions trained to within +0.0088 (Adam) and +0.0043 (SGD) of the sweep's best, inside or at the edge of the flat valley.

The steepest rule failed with Adam. Two of its five suggestions trained well (0.3278, 0.3272), but three were far too high, the worst +0.4491 above the best. With SGD it happened to work (+0.0055): its steepest points all fell between 1.75e-01 and 9.33e-01, near SGD's good range.

WHY CHOOSE THIS?

Checked against 78 real runs, the min / 10 rule gave a good rate at a tiny fraction of the cost.

WHY NOT THIS? (TRADEOFFS)

The check used 5 epochs; the best rate for a much longer run can be lower. The finder knows nothing about the budget.

4. Does the finder follow a change of batch size?

DEFINITION — start here, no prior knowledge assumed

Larger batches give less noisy gradients, so they usually tolerate, and need, a larger learning rate. With SGD, the finder and the full check are repeated at batches of 32 and 512 (3 finder runs, 13 rates × 3 seeds, and each suggestion trained with the same 3 seeds).

REAL-WORLD SCENARIO

A recipe tested for one cake may need a different oven time for a tray of twelve; a rule of thumb from one case should be re-checked on the other.

ACTUAL OUTPUT

```
  batch  32: finder (min / 10) suggests 2.60e-01, 2.74e-01, 1.98e-01; real training there 0.3509, 0.3552, 0.3444; the
      sweep's best 3e-02 (0.3358), within 0.01 of it from 1e-02 to 3e-01
  batch 512: finder (min / 10) suggests 4.70e-01, 4.45e-01, 4.96e-01; real training there 0.3679, 0.3712, 0.3680; the
      sweep's best 1e-01 (0.3437), within 0.01 of it from 1e-01 to 1e-01
Actual output from this lesson's run.
```

What it means

At batch 32 the finder suggested about 2.60e-01, inside the sweep's good range (1e-02 to 3e-01) but at its upper edge; real training there gave 0.3509, 0.3552 and 0.3444, against the best 0.3358. At batch 512 it suggested about 4.70e-01, above the sweep's best of 1e-01 (0.3437); real training there gave about 0.3679, some 0.025 worse.

So the finder's suggestion did rise with the batch size, but not reliably to a good rate. A likely reason, not tested here, is that at batch 512 five epochs are only about 240 steps, while the finder's curve describes one model taking 300 steps as the rate keeps rising; neither says how a rate behaves over a whole training run. Treat the suggestion as a starting point to check, not an answer.

WHY CHOOSE THIS?

Re-running the finder whenever the batch size or model changes is cheap.

WHY NOT THIS? (TRADEOFFS)

Its suggestion can be off by a lot (here 0.025 in log loss); a few real runs around it are still needed.

5. What the finder costs

DEFINITION — start here, no prior knowledge assumed

The finder's cost against the sweep it partly replaces.

REAL-WORLD SCENARIO

A quick test drive against a full week of trial commutes.

ACTUAL OUTPUT

```
  one finder run: 293 steps on average (1.55 of an epoch), 0.10 s
  the check in section 3: 78 runs of 5 epochs, plus the suggestions, 38 s
Actual output from this lesson's run.
```

What it means

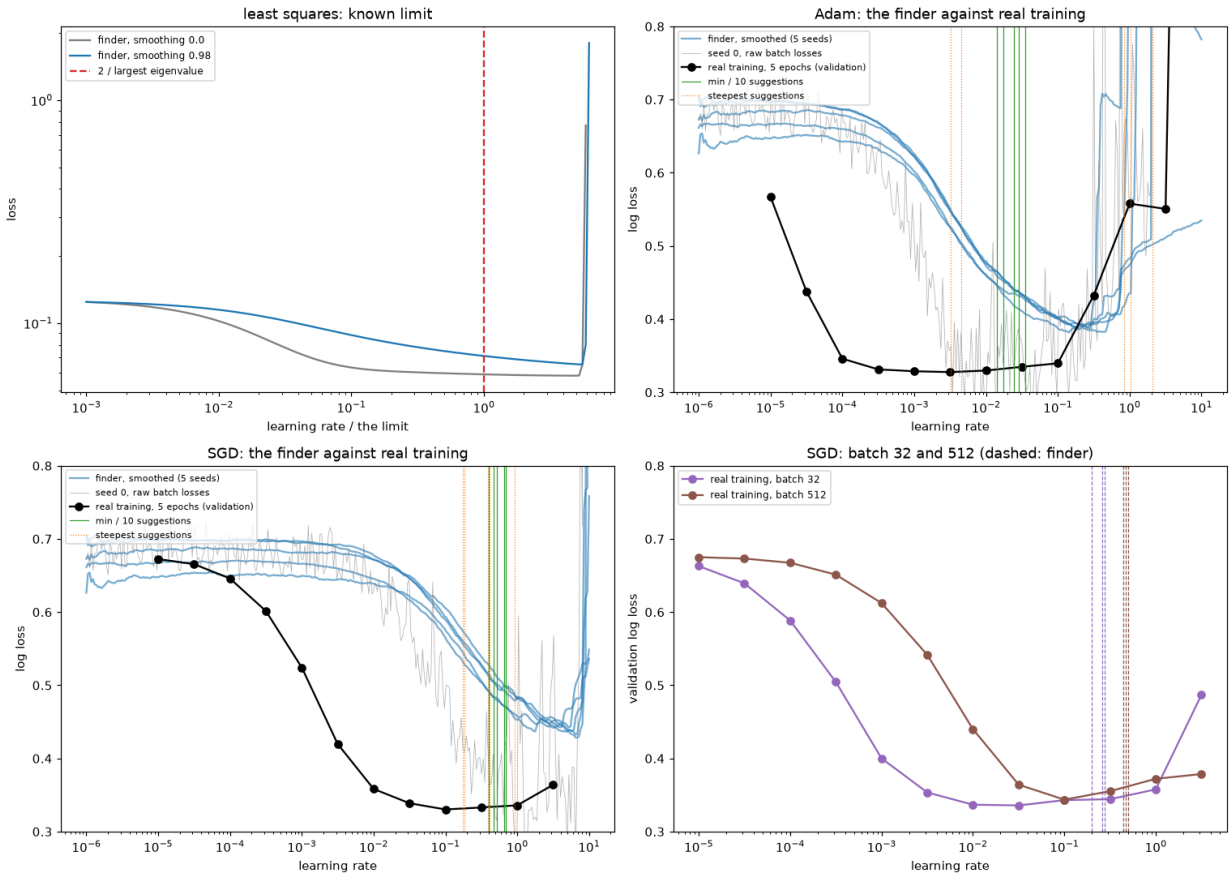
One finder run took 293 steps on average (1.55 of an epoch) and 0.10 s; the check in section 3 took 38 s. On a model that takes hours per run, that ratio is the whole point.

Today's part for the week's build

`day170_exercise.py` holds `lr_range`, `lr_find`, `suggest`, `SGD`, `bce_logits`, `train_mlp` and `mlp_stepper`, copied unchanged. A first version of its finder check used a threshold of exactly 0.1, and `lr_range` gives 0.09999999999999998 there, so the check was moved to 0.05. Its hand checks:

ACTUAL OUTPUT

```
lr_range(0.001, 1, 4): [0.001, 0.01, 0.1, 1.0] (by hand 0.001, 0.01, 0.1, 1)
lr_find, loss 1 below rate 0.05 and 10 above it, no smoothing: stopped after 3 rates at 0.1 (by
    hand 3, at 0.1: there 10 > 4 x 1; the step that blew up is kept as the last)
suggest on a bowl lowest at 0.01: min / 10 0.001 (by hand 0.001); steepest 0.001 (by hand 0.001:
    the slope is steepest at the left end, and the first 10 rates are skipped)
SGD, weight 1, gradient 0.5, rate 0.1: weight 0.950, bias -0.100 (by hand 0.95 and -0.1)
bce_logits at logit 0, label 1: loss 0.6931 (by hand ln 2 = 0.6931), gradient -0.50 (by hand 0.5 -
    1 = -0.5)
toy problem (y = 1 when x0 > 0): finder stopped at 3.29, suggests 0.227; 20 epochs there give
    validation log loss 0.141 (well below ln 2 = 0.693)
```



Top left: least squares, the finder's loss against the rate divided by the known limit. Top right and bottom left: the finder's smoothed curves (5 seeds) and one raw curve, with real training (black, validation, 5 epochs) and each run's suggestions. Bottom right: real training at batch 32 and 512, with the finder's suggestions dashed.

Interview preparation — technical questions

Q: How do you read a learning-rate finder's curve?

A: Take one tenth of the rate at the lowest smoothed loss, and look at the curve; here that rule landed within 0.009 of the best of 13 real rates.

Q: Why not use the rate at the lowest loss?

A: The finder's curve lags: on least squares its minimum was about 5 times the rate at which training diverges.

Q: Why smooth the loss, and what does it cost?

A: Single batches are noisy; but a moving average over ~50 steps also shifts the curve to higher rates.

Q: Is 'steepest slope' a good rule?

A: Not here with Adam: its suggestions moved by a factor of 645 between seeds and the worst trained 0.45 worse.

Q: How do you check a finder?

A: Real training at a grid of rates with several seeds, and at the suggestion with the same seeds.

Interview preparation — theoretical questions

Q: Why does gradient descent diverge above 2/λ?

A: Along the steepest direction each step multiplies the error by $|1 - \eta\lambda|$, which exceeds 1 there.

Q: Why does the best learning rate depend on the batch size?

A: Gradient noise falls as batches grow, so larger steps stay stable; the number of steps per epoch also falls.

Q: Does Adam remove the need to tune the learning rate?

A: No: here 5 epochs gave 0.3277 at 3e-03 but 0.5669 at 1e-05 and 3.0660 at 10, though its good valley was wide.

Q: What does the finder not tell you?

A: How a rate behaves over a long run; schedules and the training budget change the best rate.

Key takeaway and what's next

> **KEY TAKEAWAY**
> A learning-rate finder is a cheap first look, not a measurement. Its curve lags the true limit, so divide its minimum by ten; read it with a stable rule rather than the steepest slope; and confirm with a few real runs, especially after changing the batch size.

- Least squares: the finder's minimum at 5.54 times the limit; min / 10 at 0.554 times.
- Adult network: min / 10 within +0.0088 (Adam) and +0.0043 (SGD) of the best real rate.
- Steepest rule with Adam: a factor of 644.7 between seeds, worst +0.4491.
- Batch 512: suggestion trained about 0.025 worse than the sweep's best (0.3679 against 0.3437).

Suggested watch order, if starting from zero

- 1. Choosing the Learning Rate with LR Finder (Apache MXNet) — the finder, explained and demonstrated.
- 2. Competition Winning Learning Rates (Leslie Smith, MLconf) — the method's author on choosing learning rates.

Day 171 preview

- **Early stopping.** Watch the validation loss while training and stop when it stops improving: patience, restoring the best weights, and what each rule costs and saves.